

OneKhusa Node.js Integration Reference

Hosted Checkout, Disbursements & Automation

Onekhusa Team

May 2026

1 Introduction

This documentation provides a technical reference for integrating the OneKhusa Gateway. It features the **Hosted Checkout** flow for collections and a robust **Disbursement** engine for single and batch payouts, powered by real-time **Socket.io** updates.

2 Project Directory Structure

The following structure ensures a clean separation between API logic and server configuration.

```
onekhusa-nodejs-integration/
├── .env ..... API Credentials & Settings
├── .gitignore ..... Excludes .env from Git
├── src/
│   ├── app.js ..... Express Server & Webhooks
│   ├── disbursement.js ..... Payout Service
│   ├── utils.js ..... Idempotency Utility
│   └── services/
│       └── onekhusa.service.js ..... Checkout Logic
└── public/
    └── index.html ..... Main Dashboard UI
```

3 Environment Configuration (.env)

The `.env` file must be created in the root directory. It contains sensitive credentials and mapping for the NGrok public tunnel.

```
1 # OneKhusa API Credentials
2 ONEKHUSA_API_KEY=sandbox_vDdLumyfnCWiN...
3 ONEKHUSA_API_SECRET=_fefXaN9LGorR4uxlnT...
4 ONEKHUSA_ORG_ID=0BBNREQ33RSX
5 ONEKHUSA_MERCHANT_NUMBER=79619974
6 ONEKHUSA_CAPTURED_BY=admin@example.com
7
8 # API Endpoints
9 ONEKHUSA_BASE_URL=https://api.onekhusa.com/sandbox/v1
10 ONEKHUSA_CHECKOUT_URL=https://api.onekhusa.com/sandbox/v1/checkout/rtp/initiate
11
12 # Local Tunnel (Use NGrok URL)
13 PUBLIC_CALLBACK_URL=https://zena-unjudgeable-renita.ngrok-free.dev
14
15 PORT=3000
```

Listing 1: .env Template

4 Idempotency Utility

Financial requests require a unique X-Idempotency-Key to prevent duplicate processing during network retries.

```
1 const crypto = require('crypto');
2 const generateIdempotencyKey = (merchantNumber) => {
3   const uuid = crypto.randomUUID();
4   return `${merchantNumber}-${uuid}`.replace(/[~a-zA-Z0-9-]/g, "");
5 };
6 module.exports = { generateIdempotencyKey };
```

Listing 2: src/utils.js

5 Hosted Checkout Integration

The service generates a paymentTransactionId and returns a redirect URL to the frontend.

```
1 const axios = require('axios');
2 const { generateIdempotencyKey } = require('../utils');
3
4 async function initiateHostedCheckout(amount, reference, description) {
5   const payload = {
6     authentication: { apiKey: process.env.ONEKHUSA_API_KEY, apiSecret: process.
7       env.ONEKHUSA_API_SECRET },
8     merchant: { organisationId: process.env.ONEKHUSA_ORG_ID,
9       merchantAccountNumber: parseInt(process.env.ONEKHUSA_MERCHANT_NUMBER) },
10    payment: { sourceReferenceNumber: reference, description, amount: parseFloat(
11      amount) },
12    route: {
13      successRedirectionUrl: `${process.env.PUBLIC_CALLBACK_URL}/?ref=${{
14        reference}`,
15      failureRedirectionUrl: `${process.env.PUBLIC_CALLBACK_URL}/?ref=${{
16        reference}`,
17      callbackApiUrl: `${process.env.PUBLIC_CALLBACK_URL}/webhooks/payments`
18    }
19  };
20   const res = await axios.post(process.env.ONEKHUSA_CHECKOUT_URL, payload, {
21     headers: { 'X-Idempotency-Key': generateIdempotencyKey(process.env.
22       ONEKHUSA_MERCHANT_NUMBER) }
23   });
24   return res.data;
25 }
```

Listing 3: src/services/onekhusa.service.js

6 Webhook Automation

The webhook listener identifies transactions using the **TitleCase** metadata keys (**ReferenceNumber**) provided by the OneKhusa Sandbox.

```
1 app.post('/webhooks/payments', (req, res) => {
2   const payload = req.body;
3   const myRef = payload.metadata?.ReferenceNumber || payload.sourceReferenceNumber;
4
5   if (payload.responseCode === "S100" || payload.transactionStatusCode === "S") {
6     ticketTracker[myRef] = "Paid";
7     global.io.emit('webhook_received', { reference: myRef, status: 'Paid' });
8   }
9   res.status(200).send("acknowledged");
10 });
```

Listing 4: Webhook Logic